

vertices it leaves and enters, along with its weight. Argue that the Bellman-Ford algorithm runs in $O(VE)$ time without the constraint that $|E| = \Omega(V)$. Modify the Bellman-Ford algorithm so that it runs in $O(VE)$ time in all cases when the input graph is represented with adjacency lists.

22.1-6

Let $G = (V, E)$ be a weighted, directed graph with weight function $w : E \rightarrow \mathbb{R}$. Give an $O(VE)$ -time algorithm to find, for all vertices $v \in V$, the value $\delta^*(v) = \min \{\delta(u, v) : u \in V\}$.

22.1-7

Suppose that a weighted, directed graph $G = (V, E)$ contains a negative-weight cycle. Give an efficient algorithm to list the vertices of one such cycle. Prove that your algorithm is correct.

22.2 Single-source shortest paths in directed acyclic graphs

In this section, we introduce one further restriction on weighted, directed graphs: they are acyclic. That is, we are concerned with weighted dags. Shortest paths are always well defined in a dag, since even if there are negative-weight edges, no negative-weight cycles can exist. We'll see that if the edges of a weighted dag $G = (V, E)$ are relaxed according to a topological sort of its vertices, it takes only $\Theta(V + E)$ time to compute shortest paths from a single source.

The algorithm starts by topologically sorting the dag (see Section 20.4) to impose a linear ordering on the vertices. If the dag contains a path from vertex u to vertex v , then u precedes v in the topological sort. The DAG-SHORTEST-PATHS procedure makes just one pass over the vertices in the topologically sorted order. As it processes each vertex, it relaxes each edge that leaves the vertex. Figure 22.5 shows the execution of this algorithm.

DAG-SHORTEST-PATHS(G, w, s)

1 topologically sort the vertices of G

```

2 INITIALIZE-SINGLE-SOURCE( $G, s$ )
3 for each vertex  $u \in G.V$ , taken in topologically sorted order
4   for each vertex  $v$  in  $G.Adj[u]$ 
5     RELAX( $u, v, w$ )

```

Let's analyze the running time of this algorithm. As shown in Section 20.4, the topological sort of line 1 takes $\Theta(V + E)$ time. The call of INITIALIZE-SINGLE-SOURCE in line 2 takes $\Theta(V)$ time. The **for** loop of lines 3–5 makes one iteration per vertex. Altogether, the **for** loop of lines 4–5 relaxes each edge exactly once. (We have used an aggregate analysis here.) Because each iteration of the inner **for** loop takes $\Theta(1)$ time, the total running time is $\Theta(V + E)$, which is linear in the size of an adjacency-list representation of the graph.

The following theorem shows that the DAG-SHORTEST-PATHS procedure correctly computes the shortest paths.

Theorem 22.5

If a weighted, directed graph $G = (V, E)$ has source vertex s and no cycles, then at the termination of the DAG-SHORTEST-PATHS procedure, $v.d = \delta(s, v)$ for all vertices $v \in V$, and the predecessor subgraph G_π is a shortest-paths tree.

Proof We first show that $v.d = \delta(s, v)$ for all vertices $v \in V$ at termination. If v is not reachable from s , then $v.d = \delta(s, v) = \infty$ by the no-path property. Now, suppose that v is reachable from s , so that there is a shortest path $p = \langle v_0, v_1, \dots, v_k \rangle$, where $v_0 = s$ and $v_k = v$. Because DAG-SHORTEST-PATHS processes the vertices in topologically sorted order, it relaxes the edges on p in the order $(v_0, v_1), (v_1, v_2), \dots, (v_{k-1}, v_k)$. The path-relaxation property implies that $v_i.d = \delta(s, v_i)$ at termination for $i = 0, 1, \dots, k$. Finally, by the predecessor-subgraph property, G_π is a shortest-paths tree. ■

A useful application of this algorithm arises in determining critical paths in *PERT chart*² analysis. A job consists of several tasks. Each

task takes a certain amount of time, and some tasks must be completed before others can be started. For example, if the job is to build a house, then the foundation must be completed before starting to frame the exterior walls, which must be completed before starting on the roof. Some tasks require more than one other task to be completed before they can be started: before the drywall can be installed over the wall framing, both the electrical system and plumbing must be installed. A dag models the tasks and dependencies. Edges represent tasks, with the weight of an edge indicating the time required to perform the task. Vertices represent “milestones,” which are achieved when all the tasks represented by the edges entering the vertex have been completed. If edge (u, v) enters vertex v and edge (v, x) leaves v , then task (u, v) must be completed before task (v, x) is started. A path through this dag represents a sequence of tasks that must be performed in a particular order. A *critical path* is a *longest* path through the dag, corresponding to the longest time to perform any sequence of tasks. Thus, the weight of a critical path provides a lower bound on the total time to perform all the tasks, even if as many tasks as possible are performed simultaneously. You can find a critical path by either

- negating the edge weights and running DAG-SHORTEST-PATHS, or
- running DAG-SHORTEST-PATHS, but replacing “ ∞ ” by “ $-\infty$ ” in line 2 of INITIALIZE-SINGLE-SOURCE and “ $>$ ” by “ $<$ ” in the RELAX procedure.

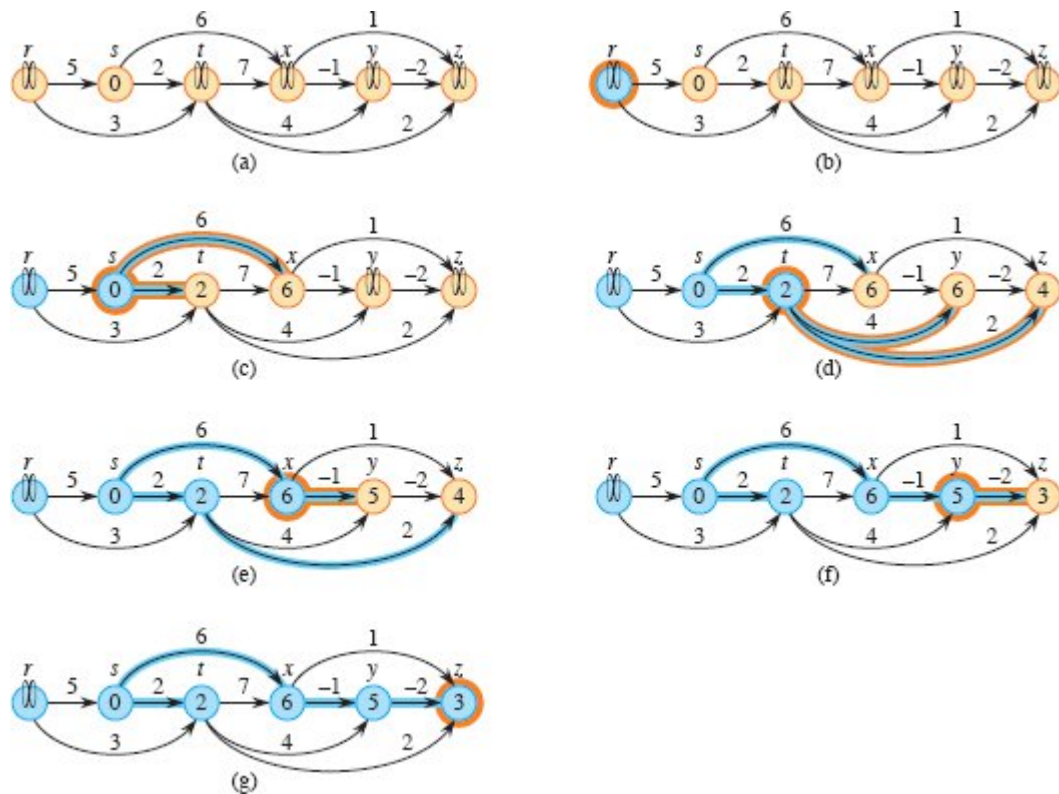


Figure 22.5 The execution of the algorithm for shortest paths in a directed acyclic graph. The vertices are topologically sorted from left to right. The source vertex is s . The d values appear within the vertices, and blue edges indicate the π values. (a) The situation before the first iteration of the **for** loop of lines 3–5. (b)–(g) The situation after each iteration of the **for** loop of lines 3–5. Blue vertices have had their outgoing edges relaxed. The vertex highlighted in orange was used as u in that iteration. Each edge highlighted in orange caused a d value to change when it was relaxed in that iteration. The values shown in part (g) are the final values.

Exercises

22.2-1

Show the result of running DAG-SHORTEST-PATHS on the directed acyclic graph of Figure 22.5, using vertex r as the source.

22.2-2

Suppose that you change line 3 of DAG-SHORTEST-PATHS to read

3 **for** the first $|V| - 1$ vertices, taken in topologically sorted order

Show that the procedure remains correct.

22.2-3

An alternative way to represent a PERT chart looks more like the dag of Figure 20.7 on page 574. Vertices represent tasks and edges represent sequencing constraints, that is, edge (u, v) indicates that task u must be performed before task v . Vertices, not edges, have weights. Modify the DAG-SHORTEST-PATHS procedure so that it finds a longest path in a directed acyclic graph with weighted vertices in linear time.

★ 22.2-4

Give an efficient algorithm to count the total number of paths in a directed acyclic graph. The count should include all paths between all pairs of vertices and all paths with 0 edges. Analyze your algorithm.

22.3 Dijkstra's algorithm

Dijkstra's algorithm solves the single-source shortest-paths problem on a weighted, directed graph $G = (V, E)$, but it requires nonnegative weights on all edges: $w(u, v) \geq 0$ for each edge $(u, v) \in E$. As we shall see, with a good implementation, the running time of Dijkstra's algorithm is lower than that of the Bellman-Ford algorithm.

You can think of Dijkstra's algorithm as generalizing breadth-first search to weighted graphs. A wave emanates from the source, and the first time that a wave arrives at a vertex, a new wave emanates from that vertex. Whereas breadth-first search operates as if each wave takes unit time to traverse an edge, in a weighted graph, the time for a wave to traverse an edge is given by the edge's weight. Because a shortest path in a weighted graph might not have the fewest edges, a simple, first-in, first-out queue won't suffice for choosing the next vertex from which to send out a wave.

Instead, Dijkstra's algorithm maintains a set S of vertices whose final shortest-path weights from the source s have already been determined. The algorithm repeatedly selects the vertex $u \in V - S$ with the minimum shortest-path estimate, adds u into S , and relaxes all edges leaving u . The procedure DIJKSTRA replaces the first-in, first-out